
TRÍ TUỆ NHÂN TẠO

Dữ liệu dạng danh mục

Lý thuyết — One-hot, OOV, hashing, feature crosses

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Giới thiệu — Dữ liệu danh mục và mã hóa (encoding)	4
1.1 Số cũng có thể là dữ liệu danh mục	4
1.2 Mã hóa (encoding)	4
2 Từ vựng (vocabulary) và mã hóa one-hot	4
2.1 Vocabulary	5
2.2 Gán số chỉ mục (index numbers)	5
2.3 One-hot encoding	6
2.4 Sparse representation	7
2.5 Outliers trong dữ liệu danh mục (OOV bucket)	7
2.6 Danh mục nhiều chiều (high-dimensional) và embeddings/hashing	7
3 Các vấn đề thường gặp với dữ liệu danh mục	8
3.1 Người gán nhãn (human raters): gold labels	8
3.2 Máy gán nhãn (machine raters): silver labels	9
3.3 High dimensionality	9
4 Tính năng kết hợp (feature crosses)	9
4.1 Khi nào dùng feature crosses	10
5 Bài tập (Playground) — Feature crosses	10
5.1 Bài tập 1: Feature cross cơ bản (XOR)	11
5.2 Bài tập 2: Feature cross “phức tạp” hơn (Circle)	11
6 Kết luận	12

Lời nói đầu

Nguồn và cách đọc

Tài liệu biên soạn và dịch đầy đủ từ *Categorical Data* (Google Machine Learning Crash Course)^a, tương ứng năm phần trong thư mục này. Mục tiêu là giữ đầy đủ ý, ví dụ, bảng và bài tập, tránh tóm tắt quá mức.

^a<https://developers.google.com/machine-learning/crash-course/categorical-data>

Thời lượng ước tính

Khoảng **50 phút** theo MLCC.

Mục tiêu học tập

- Phân biệt dữ liệu danh mục với dữ liệu số.
- Biểu diễn dữ liệu danh mục bằng **one-hot vectors**.
- Nhận biết các vấn đề thường gặp của dữ liệu danh mục.
- Tạo **feature crosses** (tính năng kết hợp).

Điều kiện tiên quyết

Giả định bạn đã học:

- [Introduction to Machine Learning](#)
- [Working with numerical data](#)

1 Giới thiệu — Dữ liệu danh mục và mã hóa (encoding)

Dữ liệu danh mục (categorical data)

Dữ liệu danh mục có một tập hữu hạn các giá trị có thể có. Ví dụ:

- Các loài động vật trong vườn quốc gia
- Tên các con đường trong một thành phố
- Email có phải spam hay không
- Màu sơn ngoại thất của nhà
- Các số đã được **bucket** (đã học ở dữ liệu số)

1.1 Số cũng có thể là dữ liệu danh mục

Ý nghĩa & Trực giác

Dữ liệu số “đúng nghĩa” thường có thể nhân/chia theo nghĩa toán học. Ví dụ, mô hình dự đoán giá nhà theo diện tích: mọi thứ khác như nhau, nhà 200 m² thường xấp xỉ gấp đôi nhà 100 m².

Lưu ý

Nhiều khi bạn nên coi một feature có giá trị nguyên là **danh mục** thay vì số. Ví dụ mã bưu điện là số nguyên; nếu coi là số, bạn đang yêu cầu mô hình tìm quan hệ số học giữa các mã: xem 20004 “gấp đôi” 10002. Nếu coi là danh mục, mô hình sẽ học trọng số cho từng mã riêng.

1.2 Mã hóa (encoding)

Encoding

Encoding là chuyển dữ liệu danh mục (hoặc dữ liệu khác) thành **vector số** để mô hình học. Cần vì mô hình chỉ huấn luyện trên **giá trị đầu thấy động**; không thể train trực tiếp trên chuỗi như "dog" hay "maple". Phần này trình bày các cách mã hóa dữ liệu danh mục.

2 Từ vựng (vocabulary) và mã hóa one-hot

Chiều (dimension)

Dimension đồng nghĩa với số phần tử trong **feature vector**.

Một số feature danh mục ít chiều

Feature	Số danh mục	Ví dụ
snowed_today	2	True, False
skill_level	3	Beginner, Practitioner, Expert
season	4	Winter, Spring, Summer, Autumn
day_of_week	7	Monday, Tuesday, Wednesday
planet	8	Mercury, Venus, Earth

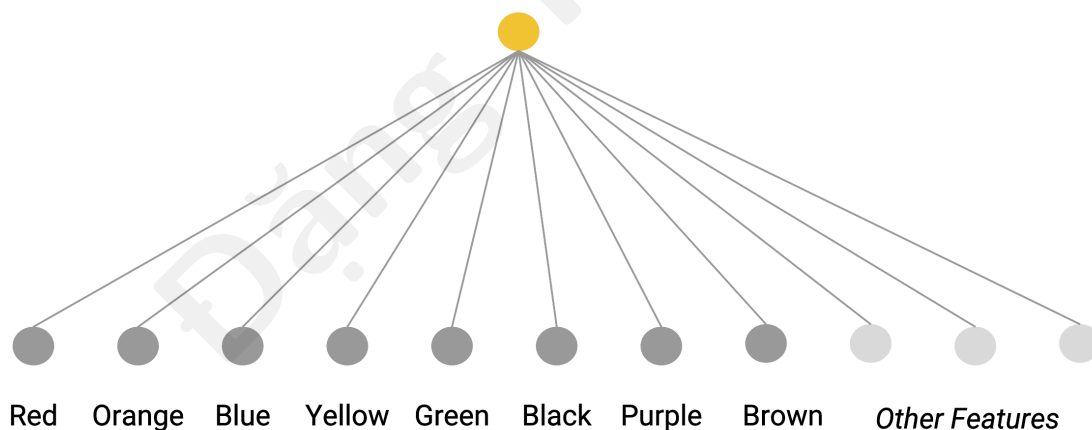
2.1 Vocabulary

Ý nghĩa & Trực giác

Khi feature danh mục có ít giá trị có thể có, ta có thể mã hóa bằng **vocabulary**: mô hình coi mỗi giá trị danh mục như một **feature riêng**. Khi huấn luyện, mô hình học trọng số khác nhau cho từng danh mục.

Feature car_color

Giả sử dự đoán giá xe, trong đó có feature danh mục `car_color`. Có thể xe màu đỏ đắt hơn xe màu xanh. Vì số màu ngoại thất hữu hạn, `car_color` là feature ít chiều. Hình sau gợi ý các giá trị trong vocabulary:



Hình 1. Mỗi màu là một feature riêng trong feature vector.

2.2 Gán số chỉ mục (index numbers)

Lưu ý

Mô hình ML chỉ thao tác được trên số (float). Vì vậy bạn phải đổi mỗi chuỗi thành một số nguyên chỉ mục duy nhất:

Ý nghĩa & Trực giác

Tuy nhiên, nếu giữ nguyên các chỉ mục như số liên tục và đưa thẳng vào mô hình, mô hình sẽ hiểu sai: có thể coi “purple” lớn gấp 6 lần “orange”. Ta cần bước tiếp theo: **one-hot**.

2.3 One-hot encoding**One-hot encoding**

Trong one-hot:

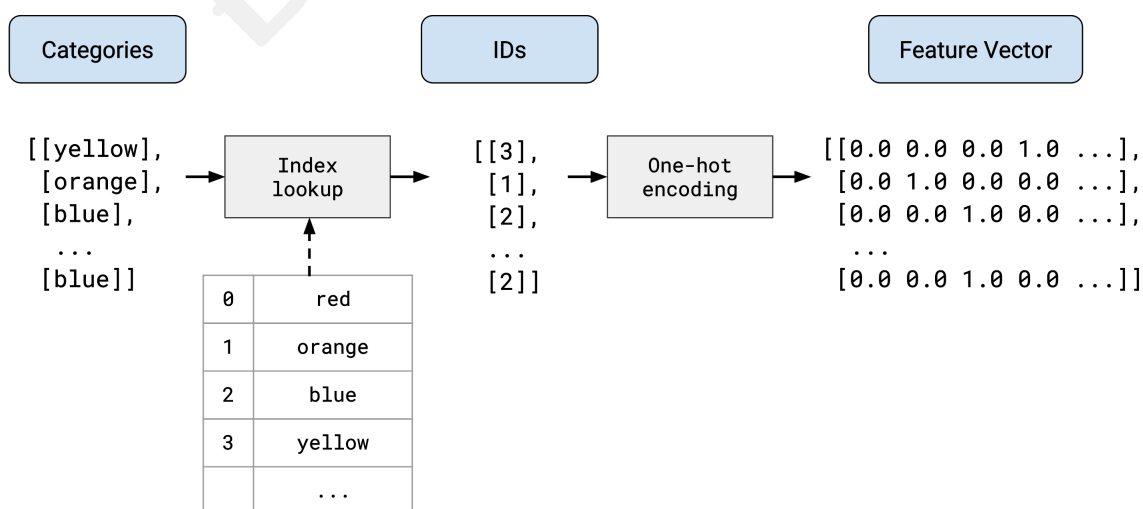
- Mỗi danh mục được biểu diễn bởi một vectơ N phần tử, N là số danh mục.
- Đúng **một** phần tử bằng 1,0; các phần tử còn lại bằng 0,0.

Chính one-hot vector (không phải chuỗi hay index) mới được đưa vào feature vector; mô hình học trọng số riêng cho từng phần tử.

Giá trị	Red	Orange	Blue	Yellow	Green	Black	Purple	Brown
“Red”	1	0	0	0	0	0	0	0
“Orange”	0	1	0	0	0	0	0	0
“Blue”	0	0	1	0	0	0	0	0
“Yellow”	0	0	0	1	0	0	0	0
“Green”	0	0	0	0	1	0	0	0
“Black”	0	0	0	0	0	1	0	0
“Purple”	0	0	0	0	0	0	1	0
“Brown”	0	0	0	0	0	0	0	1

Lưu ý

Trong one-hot “chuẩn”, chỉ một phần tử bằng 1. Một biến thể gọi là **multi-hot** cho phép nhiều phần tử bằng 1.



Hình 3. Chuỗi → index → one-hot feature vector.

2.4 Sparse representation

Sparse feature và sparse representation

Feature mà phần lớn giá trị là 0 (hoặc rỗng) gọi là **sparse feature**. **Sparse representation** là lưu **vị trí** của các phần tử khác 0 trong vectơ sparse.

Ví dụ

One-hot của “Blue”: [0, 0, 1, 0, 0, 0, 0, 0]. Nếu đếm từ 0 thì số 1 ở vị trí 2, nên sparse representation là **2**. Cách này tốn ít bộ nhớ hơn rất nhiều, nhưng lưu ý: mô hình phải **train trên one-hot vector**, không phải trên sparse representation.

Lưu ý

Với multi-hot, sparse representation lưu tất cả vị trí khác 0. Ví dụ xe vừa “Blue” vừa “Black” thì sparse representation là 2, 5.

2.5 Outliers trong dữ liệu danh mục (OOV bucket)

Ý nghĩa & Trực giác

Dữ liệu danh mục cũng có outlier. Ví dụ `car_color` ngoài các màu phổ biến còn có màu hiếm như “Mauve” hoặc “Avocado”. Thay vì dành riêng một danh mục cho từng màu hiếm, ta có thể gom chúng vào một danh mục “bất hết” gọi là **out-of-vocabulary (OOV)**. Hệ thống sẽ học một trọng số chung cho OOV bucket.

2.6 Danh mục nhiều chiều (high-dimensional) và embeddings/hash-ing

Một số feature danh mục rất nhiều giá trị

Feature	Số danh mục	Ví dụ
<code>words_in_english</code>	~500,000	“happy”, “walking”
<code>US_postal_codes</code>	~42,000	“02114”, “90301”
<code>last_names_in_Germany</code>	~850,000	“Schmidt”, “Schneider”

Lưu ý

Khi số danh mục lớn, one-hot thường là lựa chọn tệ. **Embeddings** (xem module Embeddings) thường tốt hơn vì giảm số chiều, giúp:

- Train nhanh hơn
- Suy luận nhanh hơn (latency thấp hơn)

Hashing (hashing trick)

Hashing là cách giảm số chiều ít phổ biến hơn. Tóm tắt quy trình:

1. Chọn số bin N nhỏ hơn tổng số danh mục (ví dụ $N = 100$).
2. Chọn hàm băm (và miền giá trị băm).
3. Băm từng danh mục thành số nguyên (ví dụ 89237).
4. Lấy chỉ mục = hash mod N ($89237 \% 100 = 37$).
5. Tạo one-hot trên N bin theo chỉ mục mới.

Bài tập: mô hình có thể train trực tiếp trên chuỗi?

Khẳng định: “Mô hình ML có thể train trực tiếp trên giá trị chuỗi thô như Red, Black mà không cần đổi sang vectơ số.”

Đáp án: Sai. Trong huấn luyện, mô hình chỉ thao tác được trên số thực; chuỗi không phải là số thực. Phải chuyển chuỗi sang vectơ số trước khi train.

3 Các vấn đề thường gặp với dữ liệu danh mục

Ý nghĩa & Trực giác

Dữ liệu số thường do thiết bị khoa học hoặc đo tự động ghi lại. Dữ liệu danh mục, ngược lại, thường do con người phân loại hoặc do mô hình ML phân loại. Ai quyết định danh mục/nhãn và cách quyết định ảnh hưởng đến độ tin cậy của dữ liệu.

3.1 Người gán nhãn (human raters): gold labels

Gold labels

Dữ liệu do người gán nhãn thủ công thường gọi là **gold labels** và thường được ưa chuộng hơn dữ liệu gán nhãn bằng máy vì chất lượng tương đối tốt hơn.

Lưu ý

Tuy vậy, không phải cứ người gán nhãn là dữ liệu sẽ tốt. Lỗi, thiên kiến, thậm chí ác ý có thể được đưa vào khi thu thập hoặc khi làm sạch/tiền xử lý. Cần kiểm tra trước khi train.

Inter-rater agreement

Hai người có thể gán nhãn khác nhau cho cùng một ví dụ. Mức độ khác nhau giữa các người gán nhãn gọi là **inter-rater agreement**. Bạn có thể dùng nhiều người gán nhãn cho mỗi ví dụ và đo inter-rater agreement để ước lượng độ biến thiên.

Một số thước đo inter-rater agreement

- Cohen's kappa và các biến thể
- Intra-class correlation (ICC)
- Krippendorff's alpha

Tài liệu tham khảo: Hallgren 2012 (Cohen's kappa, ICC) và Krippendorff 2011 (alpha).

3.2 Máy gán nhãn (machine raters): silver labels**Silver labels**

Dữ liệu gán nhãn tự động bằng một hoặc nhiều mô hình phân loại thường gọi là **silver labels**. Chất lượng có thể dao động rất lớn.

Lưu ý

Cần kiểm tra không chỉ độ chính xác mà cả thiên kiến và những vi phạm “thường thức”/ý định. Ví dụ mô hình thị giác nhầm ảnh **chihuahua** thành **muffin** và ngược lại; hoặc mô hình cảm xúc cho từ trung tính điểm -0,25 (trong khi 0,0 là trung tính) $\Rightarrow$ thiên kiến âm. Bộ lọc độc hại quá nhạy có thể gán cờ sai nhiều câu trung tính. Hãy đánh giá chất lượng và thiên kiến của nhãn máy trước khi train.

3.3 High dimensionality**Ý nghĩa & Trực giác**

Dữ liệu danh mục có xu hướng tạo feature vector **rất nhiều chiều**. Nhiều chiều làm tăng chi phí huấn luyện và khiến việc học khó hơn, nên thường cần giảm số chiều trước khi train. Với ngôn ngữ tự nhiên, cách chính là chuyển sang **embedding vectors** (xem module Embeddings).

4 Tính năng kết hợp (feature crosses)**Feature cross**

Feature crosses được tạo bằng cách lấy **tích Descartes** của hai hoặc nhiều feature danh mục (hoặc đã bucket). Tương tự **polynomial transforms** ở dữ liệu số, feature crosses giúp mô hình tuyến tính xử lý phi tuyến và mã hóa **tương tác** giữa feature.

Bộ dữ liệu lá cây

Hai feature danh mục:

- edges: smooth, toothed, lobed

- arrangement: opposite, alternate

Giả sử one-hot theo thứ tự cột trên, lá có smooth và opposite được biểu diễn là $\{(1, 0, 0), (1, 0)\}$.

Tích Descartes

Feature cross của hai feature trên tạo tập danh mục:

$\{\text{Smooth_Opposite}, \text{Smooth_Alternate}, \text{Toothed_Opposite}, \text{Toothed_Alternate}, \text{Lobed_Opposite}, \text{Lobed_Alternate}\}$

Mỗi phần tử là tích các phần tử gốc, ví dụ: $\text{Smooth_Opposite} = \text{edges}[0] \cdot \text{arrangement}[0]$, $\text{Lobed_Alternate} = \text{edges}[2] \cdot \text{arrangement}[1]$, v.v. Nếu lá có lobed và alternate thì vectơ feature-cross là $\{0, 0, 0, 0, 0, 1\}$.

Lưu ý

Dữ liệu này có thể dùng để phân loại lá theo loài cây vì các đặc điểm đó ổn định trong cùng loài.

So sánh nhanh với polynomial transforms

Cả hai đều tạo **synthetic feature** để mô hình tuyến tính học phi tuyến. Polynomial transforms thường kết hợp dữ liệu số; feature crosses kết hợp dữ liệu danh mục.

4.1 Khi nào dùng feature crosses

Lưu ý

Kiến thức miền (domain knowledge) thường gợi ý cách kết hợp hữu ích. Nếu không có, khó chọn feature cross thủ công. Có thể dùng **neural networks** để tự tìm tổ hợp hữu ích khi huấn luyện (nhưng tốn tính toán).

Cẩn thận: cross làm bùng nổ chiều và càng sparse

Cross hai sparse feature tạo feature mới còn sparse hơn. Nếu A là sparse 100 chiều và B là sparse 200 chiều, cross A và B tạo sparse 20,000 chiều.

5 Bài tập (Playground) — Feature crosses

Ý nghĩa & Trực giác

Playground là ứng dụng tương tác cho phép thay đổi feature, siêu tham số và quan sát ảnh hưởng lên mô hình. Trang này có hai bài tập.

5.1 Bài tập 1: Feature cross cơ bản (XOR)

Giao diện cần chú ý

Dưới mục FEATURES có ba feature: x_1 , x_2 , và x_1x_2 . Dưới mục OUTPUT là một hình vuông với chấm cam/xanh: tương ứng rừng vuông, chấm cam là cây bệnh, chấm xanh là cây khỏe. Giữa FEATURES và OUTPUT có ba đường đứt nối feature $\rightarrow$ output; độ dày biểu diễn trọng số. Ban đầu các đường rất mờ vì trọng số khởi tạo 0.

Task 1

Khám phá Playground:

1. Bấm vào đường mờ nối x_1 với output (hiện popup).
2. Nhập trọng số 1.0.
3. Nhấn Enter.

Quan sát:

- Đường đứt của x_1 dày hơn (trọng số $0 \rightarrow 1.0$).
- Nền cam/xanh xuất hiện: nền cam là dự đoán nơi cây bệnh; nền xanh là dự đoán nơi cây khỏe.
- Mô hình rất tệ; khoảng một nửa dự đoán sai.
- Vì trọng số $x_1 = 1$ và các feature khác 0, mô hình khớp đúng theo x_1 .

Task 2

Thay đổi trọng số của một hoặc cả ba feature để nền màu dự đoán đúng cây bệnh/cây khỏe.

Lời giải gợi ý: đặt $w_1 = 0$, $w_2 = 0$, và trọng số của x_1x_2 là một số dương bất kỳ.

Lưu ý

Thử nhập giá trị **âm** cho feature cross thì điều gì xảy ra?

5.2 Bài tập 2: Feature cross “phức tạp” hơn (Circle)

Task 1

Chạy mô hình và quan sát:

1. Bấm Run/Pause (tam giác trắng trong vòng tròn đen) để train; quan sát Epochs tăng.
2. Train ít nhất 300 epoch rồi Pause.
3. Mô hình dự đoán tốt không? (chấm xanh nằm trong nền xanh, chấm cam trong nền cam?)
4. Xem Test loss (dưới OUTPUT): gần 1.0 hay gần 0?

5. Reset bằng nút mũi tên cong.

Lời giải: mô hình rất tệ; nhiều chấm cam nằm trong nền xanh; Test loss rất cao.

Task 2

Tạo mô hình tốt hơn:

1. Chọn/bỏ chọn kết hợp bất kỳ trong 5 feature.
2. Điều chỉnh learning rate.
3. Train ít nhất 500 epoch.
4. Quan sát Test loss: có đạt < 0.2 không?

Lời giải gợi ý:

- Chọn hai polynomial transforms (x_1^2 và x_2^2), bỏ chọn ba feature còn lại.
- Giảm learning rate xuống 0.001 hoặc thấp hơn.

Tìm hiểu thêm

playground.tensorflow.org

6 Kết luận

Tổng kết chương

- Dữ liệu danh mục cần **encoding** trước khi train vì mô hình chỉ nhận float.
- Với ít danh mục: vocabulary + one-hot; chú ý sparse, OOV bucket.
- Với nhiều danh mục: thường dùng **embeddings** hoặc **hashing** để giảm số chiều.
- Dữ liệu nhãn do người/máy có thể sai hoặc lệch: cần đánh giá chất lượng nhãn, đo inter-rater agreement.
- Feature crosses giúp mô hình tuyến tính học tương tác/phi tuyến, nhưng dễ bùng nổ chiều.

Tiếp theo: **Datasets, generalization, and overfitting**.